

// kcl cyber security society

patch wednesday

wed 28 oct · 13:00–15:00 · bush house (s) 6.03

last week you broke
things. today you
fix them.

two sides of the same bug

red team

find the hole and walk through it. that was welcome ctf.

blue team

close the hole so no one can. that's today.

a real fix kills the **whole class** of attack, not the one payload you were shown. a blocklist of "bad words" is not a fix.

the loop

1. grab your own copy of the repo (**use this template**)
2. find a bug, read the hack in the file
3. write the fix
4. git push → github runs the tests
5. red cross → keep going. **all green → patched.**

no python needed. github runs everything. your copy starts **red** on purpose.

two things

1. a **github account** — github.com/signup
2. **git installed** — git-scm.com (windows: default options)

```
$ git --version
git version 2.x  # anything prints = you're good
```

stuck? pair up with the person next to you and carry on.

git-scm.com

fork-free: use this template

1. open the repo (qr) → green **use this template** → **create**. keep it **public**.

2. on your new repo: green **code** button → copy the https link.

3. in a terminal (windows: **git bash**):

```
$ git clone <your link>
$ cd <your repo>
```

then open the folder in vs code.

the repo

bug #1: the login query

challenges/login.py — the query is built from what you type

```
row = con.execute(
  "... WHERE username = '" + username + "'
       AND password = '" + password + "'"
)
```

type a username that ends the string early and comments out the password check, and you're in without a password. that's **sql injection**.

why a blocklist fails

"just ban the dashes and ORs"

```
if "--" in username or " or " in username:
    return None
```

push it → still **red**. the test tries another shape ('/*, a different comment) and walks right past your list. you can't enumerate every trick.

stop mixing input into the query

```
row = con.execute(
  "... WHERE username = ? AND password = ?",
  (username, password),
)
```

the ? placeholders keep your input as **data**, never as part of the query. push → **green**. that's the pattern for the whole class.

```
$ git commit -am "parameterise the login query"
$ git push
```

now you: two bugs (+ two bonus)

notes.py — read anyone's notes by changing the url number (idor)

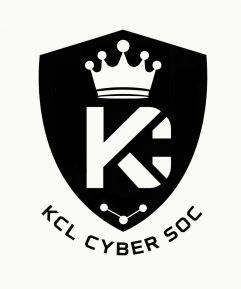
files.py — the download escapes its folder (path traversal)

tools.py — bonus: the host box runs extra commands

redirect_after_login.py — bonus: ?next= jumps off-site

each file tells you the bug and the fix. read the tests to see what they check. hints are free — shout.

done early? swap repo links with a neighbour and break their fix. then hunt: at least 3 more bugs the tests don't check.



all green?

you just patched five real classes of web bug, and shipped each fix the way a team does: push, tests, merge.

keep in touch — discord & everything else at kclcybersoc.org.uk

kclcybersoc.org.uk